

~/ profile-readme.guide

Build a GitHub profile that **isn't** a wall of badges.

Every step I took to make mine — the magic repo, the dot-matrix portrait, the charts that draw themselves, and the three robots that keep it all current while I sleep.

Gargi Bhardwaj

github.com/gargibhardwaj24

a walkthrough, not a template dump

01100010 01110101 01101001 01101100 01100100 00100000 01110011 01101111 01101101 01100101 01110100 01101000 01101001 01
101110 01100111 00100000 01110111 01101111 01110010 01110100 01101000 00100000 01101100 01101111 01101111 01101011 0110
1001 01101110 01100111 00100000 01100001 01110100

What you're actually building

Every profile README I opened looked the same: a waving-hand emoji, twelve skill badges in a row, and three stat cards that were broken. I wanted mine to look like something I'd built rather than something I'd pasted. This is the whole recipe.

Here's what you end up with, top to bottom — and every single piece of it is a file that lives in **your** repo, so nothing on this page can go down when someone else's server does.

Portrait + typing banner + social badges

a photo turned into a dot matrix, your name typed out live
[dotify.py](#) · [readme-typing-svg](#) · [shields.io](#)

whoami + toolbox

four honest bullets and a strip of stack icons

Two radar charts

one you rate by hand, one drawn from real bytes in your repos
[radar.py](#)

Contribution calendar + snake

3D isometric year, then a snake that eats it
[lowlighter/metrics](#) · [Platane/snk](#)

Stat card + language mix + achievements

self-hosted, so they never 503 on you
[cards.py](#) · [lowlighter/metrics](#)

Four project cards

live stars, forks and language, refreshed daily

GREEN = REGENERATES ITSELF

Everything shaded green is rebuilt on a schedule by GitHub Actions. You write the config once and never touch it again.

WHAT YOU NEED

- a GitHub account
- Python 3.10 or newer
- one photo of yourself
- git, and a terminal
- about 45 minutes

WHAT YOU DON'T NEED

- a server
- a paid anything
- design skills
- to know how SVG works

How to read this

Nine steps, in order. Steps 0 through 5 happen on your own machine and you can see the results in a browser before anyone else sees anything. Steps 6 and 7 put it on GitHub and switch on the automation. Step 8 is the list of things that will go wrong, because a few of them will. Step 9 is the part that actually makes it yours, and it's the one I'd skip if I were you — so please don't.

ONE HONEST WARNING

The temptation is to clone this, swap the username, and ship it. It'll work, and it'll also look like my profile with your name on it. The generators are the reusable part; the taste isn't. Step 9 tells you exactly which bits to tear out.

The magic repo

None of this works until you know the one trick GitHub never tells you about.

Create a repository named **exactly the same as your GitHub username**. Not similar. Identical. If your username is `anaya-writes`, the repo is `anaya-writes`. GitHub treats that one repo as special: its `README.md` is rendered at the top of your profile page, above the pinned repos, for everyone who visits.

Go to github.com/new and:

- **Repository name** — your username. GitHub will pop up a little "🌟 You found a secret! 🌟" message the moment you type it correctly. That's your confirmation you got it right.
- **Public** — not optional, and I'll explain why in a second.
- **Add a README file** — tick it, so the repo isn't empty.

WHY PUBLIC MATTERS

Every image in this guide is a file loaded by URL. If the repo is private, *you* will see all of them because you're logged in and authorised — and everyone else gets a page of broken image icons. You will not notice this. You will be the only person who can't notice it. Make it public.

A quick sanity check before you spend an hour

Edit the README GitHub just made you, put a single line in it, commit, and open `github.com/YOUR_USERNAME`. If that line shows up at the top of your profile, the magic repo is wired correctly and everything from here is just filling it in. If it doesn't, the repo name doesn't match your username — go rename it.

STEP 1

Start from mine, then gut it

Three Python scripts do the heavy lifting. Copy those, keep the workflows, and throw away everything that's about me.

TERMINAL

```
# grab it, then immediately detach it from my history
git clone https://github.com/gargibhardwaj24/gargibhardwaj24.git my-profile
cd my-profile
rm -rf .git
git init && git branch -M main

# my face and my charts are not your face and your charts
rm -f assets/*.svg assets/jacket.png
```

Now install the one dependency:

TERMINAL

```
pip install pillow
```

That really is the only one. Everything else in the scripts is Python standard library, and that's deliberate — the GitHub Actions runners can then execute them without a `requirements.txt` install step, which is one fewer thing to break at 3am.

What you just cloned

FILE	WHAT IT'S FOR
<code>scripts/dotify.py</code>	Turns a photo into a dot-matrix SVG portrait
<code>scripts/radar.py</code>	Draws both radar charts — hand-rated and live from the API
<code>scripts/cards.py</code>	Draws the stat card and the project cards into your repo
<code>.github/workflows/</code>	Three workflows: metrics, snake, charts-and-cards
<code>assets/skills.json</code>	Your self-rated radar values — you edit this
<code>assets/projects.json</code>	Which repos get a project card — you edit this
<code>preview.html</code>	Opens in a browser, shows every asset on dark <i>and</i> light
<code>setup.ps1</code>	Windows only — fills placeholders and runs the generators in one go
<code>.gitattributes</code>	Boring, load-bearing. Leave it. Step 8 explains why.

The shortcut, if you're on Windows

There's a script that does the find-and-replace and runs all the generators for you:

```
POWERSHELL
```

```
.\setup.ps1 -Username YOUR_USERNAME -Name "Your Name" -Image .\me.jpg -Circle
```

It rewrites `YOUR_USERNAME` and `YOUR NAME` across the README and the workflows, draws the radars, and dot-matrixes your photo. You'll still fill in the links by hand. If you're on macOS or Linux, skip it and run the three Python commands in steps 3–5 directly — that's all it does anyway.

STILL TO REPLACE BY HAND

Search the README for these and swap them out: `YOUR_LINKEDIN`, `YOUR_EMAIL`, `YOUR_PORTFOLIO`, `YOUR_CF`, `YOUR_LEETCODE`, and `PROJECT_ONE` through `PROJECT_FOUR`. Then rewrite the whoami bullets, because those are mine and they're about me.

STEP 2

The header, and the one trick worth learning

A README is Markdown, but GitHub also renders a subset of HTML — which matters, because Markdown cannot centre anything and a profile that's all left-aligned looks like a changelog.

So the whole top of the file is wrapped in one tag:

```
MARKDOWN

<div align="center">

    ... portrait, banner, badges ...

</div>
```

Blank lines inside the div are important — they let GitHub keep parsing Markdown in between the HTML. Squash them and your links stop rendering.

The typing banner

This is an SVG served by a free endpoint that animates text like it's being typed. It's a URL, not a file, and every setting is a query parameter:

```
MARKDOWN


```

The bits worth knowing:

- **lines** — separated by semicolons, and this is where people get stuck. It's a URL, so **spaces become +**, and an ampersand in your text has to be **%26** or it silently truncates the whole thing.
- **color** — hex without the **#**. Mine is **39D353**, which is the brightest green square in the contribution graph. Pick yours and then use it everywhere else too.
- **duration** / **pause** — milliseconds. Slower than you think reads better; 2600 and 900 is a calm pace.
- **width** — set it wide enough for your longest line or it clips mid-word.

My last line is **01100011 01101111 01100100 01100101**, which is "code" in binary. That's a bit. Have your own bit or don't have one — see step 9.

The social badges

All of them come from shields.io and follow one pattern:

MARKDOWN

```
<a href="https://linkedin.com/in/YOUR_HANDLE"></a>
```

Read it as `badge/LABEL-BACKGROUNDHEX?logo=NAME`. Use each brand's actual hex — LinkedIn blue, Gmail red — so the row reads as a row of real logos rather than a row of your accent colour. Logo names come from **simpleicons.org**; if a badge shows up with no icon, the name you used isn't in that list.

And the view counter, which is just an image whose pixels happen to be a number:

MARKDOWN

```

```

The trick: `<picture>`

This is the single most useful thing in the whole guide, so I'm giving it its own heading.

GitHub has a dark theme and a light theme, and you don't get to know which one your visitor is using. A chart with dark grey labels is invisible on dark. A chart with white labels is invisible on light. Half your audience sees a broken-looking blank box, and you never find out because you only ever look at your own profile in your own theme.

The fix is a plain HTML tag that GitHub fully respects:

MARKDOWN

```
<picture>
  <source media="(prefers-color-scheme: dark)" srcset="assets/radar-dark.svg">
  <source media="(prefers-color-scheme: light)" srcset="assets/radar-light.svg">
  
</picture>
```

The browser picks the matching `source`; the `img` is the fallback for anything that doesn't understand the tag, and it also supplies the `alt` text and the width.

This is why every generator in this guide writes **two** files — `-dark.svg` and `-light.svg` — instead of one. Once you've seen it you'll notice how many profiles get this wrong.

HOW TO ACTUALLY TEST IT

Change your *own* GitHub theme (Settings → Appearance) and reload your profile. Thirty seconds, and it's the only way you'll ever catch a light-mode disaster.

STEP 3

The portrait

This is the piece that makes a profile look handmade instead of assembled. It's also the piece with the most knobs, so here's which ones actually matter.

`dotify.py` reads a photo, samples it down to a grid, and writes an SVG where every cell is a dot whose size tracks how bright that patch of the photo was. Drop your photo in the repo and run:

TERMINAL

```
python scripts/dotify.py me.jpg -o assets/portrait \
  --cols 100 --equalize --detail 0.5 --color
```

That's the exact command behind mine. Now here's what each flag is doing, in the order I'd care about them.

`--equalize` · the one that isn't optional

A photo of a face is a lit forehead against dark hair against a mid-tone background. That's a far wider range of brightness than the ten or so dot sizes the renderer can actually draw. Render it straight and the whole face collapses into one flat blob of big dots and the hair disappears into nothing.

`--equalize` stretches the tones against the subject's own histogram before sampling, which buys back the shadow detail. Try one render without it just once so you can see what it's fixing — then leave it on forever.

`--detail 0.5` · put the face back

Equalising flattens local structure as a side effect. This adds it back on top: cheekbones, the bridge of the nose, the edge of a jaw. Above about `1.0` it stops looking like detail and starts looking like noise. `0.5` is the sweet spot.

`--cols` · the quality dial, and the only one with a cost

How many dots across. This is your whole size-versus-detail tradeoff, and it matters more than you'd expect because the SVG has to load on someone's phone.

VALUE	RESULT	FILE SIZE
60	Chunky, abstract, reads as an idea of a person	small
88	The default. Safe.	modest
100	What I shipped — recognisably me	~325 KB
130	More detail, but now you're past 500 KB	too much

`--color` · one file instead of two

Keeps each dot's original colour from the photo. Because the fills then come from your photograph rather than from a theme, it writes a single `portrait.svg` that works on both GitHub themes — so this is the one asset that doesn't need the `<picture>` dance. Leave `--color` off and you get the green monochrome version instead, matching the contribution graph, as a `-dark / -light` pair.

`--reveal` · the slow scan

Draws the portrait in row by row when the page loads, like an image coming in over a slow connection. `--reveal-time` is the full sweep (2.5s), `--reveal-fade` is how long a single row takes to appear (0.45s — this is the number that makes it a soft scan rather than a hard wipe), and `--reveal-dir up` sweeps the other way.

It plays once per image load, so to watch it again open `preview.html` and hit the **replay the load-in** button.

the rest, briefly

`--animate`

A slow shimmer sweeping across the columns. Gorgeous on the green monochrome version. On the colour one it reads as vertical banding across your face, which is why mine doesn't use it. It stacks with `--reveal` if you want both.

`--circle`

Masks to a circle with a feathered edge. Great for a tight head shot, wrong if your framing includes shoulders — it lops them off.

`--square --focus 0.55,0.45`

Crops to 1:1, and `--focus` says which point of the original should end up in the middle. `0.55,0.45` means "the subject sits a bit right of centre and a bit high".

`--invert`

For a dark subject on a light background.

`--mode`

`dots` is the default. `binary` renders literal 0s and 1s. `ascii` and `braille` write a `.txt` you paste into a fenced code block instead.

THE TRICK THAT IMPROVED MINE MOST

Cut yourself out of the background first — any background remover will do — and save as PNG with transparency. The script treats an alpha channel as a subject mask: nothing gets drawn outside you, and more importantly `--equalize` then measures *only you* instead of averaging in a huge flat backdrop. My source photo had an orange background; that's why it simply isn't in the output.

Look at it before anyone else does

Open `preview.html` in a browser. It shows every generated asset twice, once on a dark card and once on a light one, exactly the way GitHub will render them. Nothing is pushed yet, nothing is public, and you can iterate on `--cols` and `--detail` as many times as you like.

STEP 4

The middle: whoami, toolbox, radars

whoami

Four bullets. Not ten. This is the only part of the page written in actual sentences, so it carries all the personality — and a list of twenty things you're "passionate about" carries none.

Mine is a fake terminal prompt, which is a stylistic choice you should feel free to bin:

```
MARKDOWN

## `~/` whoami

```console
$ cat about.txt
```

Hi, I'm Your Name. One or two lines about what you build and why.

- Currently building [Project](link)
- Portfolio: [yoursite.com](link)
- Learning something specific
- Fun fact: something that is actually a fun fact
```

"Currently building" is the highest-value line on the whole page and the one that goes stale fastest. Put a reminder somewhere to update it.

The toolbox strip

One image, one URL, no maintenance:

```
MARKDOWN


```

`perline` controls how many icons per row — set it so the last row isn't a lonely single icon. The full list of names is at skillicons.dev.

RESIST

Only list things you'd be happy to be asked about in an interview. A wall of thirty logos says "I have used a tutorial", which is the opposite of what you want it to say.

The two radars

One is your opinion of yourself. The other is the truth. Showing both together is the whole joke, and it's also genuinely informative.

the self-rated one

Edit `assets/skills.json` — values are 0–100 and entirely made up by you:

```
JSON

{
  "title": "Skill Radar",
  "axes": [
    { "label": "JavaScript", "value": 88 },
    { "label": "TypeScript", "value": 80 },
    { "label": "React", "value": 82 },
    { "label": "Node / API", "value": 74 },
    { "label": "Databases", "value": 70 },
    { "label": "Python", "value": 65 },
    { "label": "DSA", "value": 72 }
  ]
}
```

```
TERMINAL

python scripts/radar.py --data assets/skills.json -o assets/radar
```

Five to eight axes reads best. Past eight, the labels start colliding with each other and the shape stops being legible. And don't put everything at 90 — a radar where every axis is maxed is a circle, and a circle communicates nothing.

the honest one

This one needs no editing ever. It counts actual language bytes across your public repos and draws the result:

```
TERMINAL

python scripts/radar.py --github YOUR_USERNAME -o assets/radar-langs \
  --limit 7 --values --curve 0.4 \
  --exclude "shell,makefile,dockerfile,batchfile,procfile"
```

--curve

The important one. Raw byte counts are brutally lopsided — if one language is 90% of your code, a linear radar is just a single spike with six dots around it. `1.0` is linear, `0.5` is a square root, `0.4` is what I use, `0.3` flattens it further. It's not lying; it's a log-ish scale, same as every chart of anything that spans orders of magnitude.

--exclude

Drops languages from the count. HTML, CSS and Shell are excluded by default, because a single `dist/` folder that got committed once will otherwise make HTML your top language forever.

--limit 7

How many axes. Same 5–8 rule as above.

--values

Prints the number next to each axis. Keep it — the shape alone is decorative, the numbers make it information.

Then drop them side by side in a two-column table so they read as a pair:

MARKDOWN

```
<table><tr>
<td width="50%" align="center"> <!-- self-rated radar --> </td>
<td width="50%" align="center"> <!-- language radar --> </td>
</tr></table>
```

Tables are how you get columns in a README. There is no other way, and yes, it feels like 2004.

STEP 5

The cards — and why I stopped using the popular ones

This is the one place where I did something different from every other profile, and it's the decision I'd most want you to copy.

Everyone uses the same three things for stats: `github-readme-stats`, `github-profile-trophy`, and `streak-stats`. They're excellent projects. The problem isn't the code — it's that you're pointing at **someone else's server**, shared with hundreds of thousands of other profiles, and rendering it fresh on every single page view.

When I was building this, those three were returning, respectively: **503**, **402 — quota exhausted**, and intermittent timeouts. So the section of my profile that was supposed to say "here's what I've done" instead said, to anyone who looked, "this person's page is broken."

Your profile README is frequently the first thing a recruiter opens. Three broken images is meaningfully worse than no images.

So the cards get generated into your own repo

`cards.py` writes real SVG files into `assets/`. Stars, forks and language are pulled live from the GitHub API *when the script runs*, so the numbers genuinely track reality — they just do it on a daily schedule instead of on every page view. And because the result is a static file sitting in your repo, there is no server that can be down.

TERMINAL

```
python scripts/cards.py --user YOUR_USERNAME --out assets
```

choosing which repos get a card

That's `assets/projects.json`, in the order they'll appear:

JSON

```
{
  "projects": [
    { "repo": "dossier",
      "description": "An immersive design-engineering portfolio. Fluid scroll inertia, geometric morphing, and typography that bends toward your cursor." },
    { "repo": "Sage",
      "description": "A local-first calendar that shapes your week around six productivity frameworks." }
  ]
}
```

The `description` here overrides whatever GitHub has for that repo. That's useful because most people never set a repo description — but you should go and set them on GitHub too, since it's what anyone browsing your repo list actually sees. Once they're set, you can delete these overrides.

Write these like a one-line pitch, not like a commit message. "A local-first calendar that shapes your week around six productivity frameworks" tells me something. "React app for scheduling" does not.

three tiles or six

Total contributions and current streak come from GitHub's GraphQL API, which requires a token. Without one, the card still renders — you just get three tiles instead of six. Step 6 sets that token up, and the workflow passes it in automatically. Locally, three tiles is fine.

The project card grid

Four cards, two by two, each one wrapped in a link so the whole card is clickable:

MARKDOWN

```
<td width="50%">
  <a href="https://github.com/YOU/dossier">
    <picture>
      <source media="(prefers-color-scheme: dark)" srcset="assets/card-dossier-dark.svg">
      <source media="(prefers-color-scheme: light)" srcset="assets/card-dossier-light.svg">
      
    </picture>
  </a>
</td>
```

Underneath mine there's a small table repeating the same four projects with a live link and the stack. That's not redundant — the cards are for people scanning, the table is for people who've decided to actually look at something and want the deploy URL. Wrap it in `<sub>` so it sits quietly under the cards instead of competing with them.

PICK FOUR, NOT ELEVEN

Four cards fill a 2×2 grid cleanly and force you to choose. If your fifth project deserves a spot, something else has to lose one — and having to make that call is the useful part.

STEP 6

The three robots

Everything so far you ran by hand. From here it runs itself — three GitHub Actions workflows that wake up on a schedule, regenerate the artwork, and commit it back to your repo.

WORKFLOW	MAKES	RUNS	OUTPUT LANDS IN
<code>metrics.yml</code>	3D isometric calendar, language mix, achievement badges	every 6h	<code>assets/metrics/*.svg</code> on <code>main</code>
<code>snake.yml</code>	The snake eating your contribution graph	every 12h	a separate <code>output</code> branch
<code>radar.yml</code>	Both radars, the stat card, the project cards	daily, 03:30	<code>assets/radar/*.svg</code> , <code>assets/card-*.svg</code>

All three also have `workflow_dispatch`, which is what puts the **Run workflow** button in the Actions tab. You'll need it in step 7.

One line you must change

Two of the workflows figure out your username automatically from `${{ github.repository_owner }}`. The metrics one doesn't — it takes an explicit `user:`, four times over. Open `.github/workflows/metrics.yml` and replace every `user: gargibhardwaj24` with your own username. While you're in there, `config_timezone` is set to `Asia/Kolkata`; change it if you'd like your "commits by hour" data to reflect your actual hours.

Where the snake lives

The snake images don't go into `main`. They're pushed to an orphan branch called `output`, so the animation frames never clutter your commit history. That's why the README references them by full URL instead of a relative path:

MARKDOWN

```
https://raw.githubusercontent.com/YOU/YOU/output/snake-dark.svg
```

Swap both `YOU`s for your username. And note: this URL **404s until the snake workflow has completed once**. That's expected, not a bug — the branch doesn't exist yet.

The two settings everybody forgets

If your workflows fail on the first run, it's one of these two. It's essentially always one of these two.

A · let Actions write to your repo

Repo → Settings → Actions → General → Workflow permissions → Read and write permissions → Save.

By default, workflows can only read. But every one of these three needs to *commit files back* — that's the entire point. Without this, the radar workflow fails at the push step and the snake workflow can't create the `output` branch at all.

B · give the metrics workflow its own token

`lowlighter/metrics` reads profile-level data — your contribution calendar, your achievements — and the token GitHub hands to every workflow automatically can't see any of that. It needs a personal access token of your own.

1. Go to **github.com/settings/tokens → Generate new token (classic)**.
2. Scope: tick `read:user`. Add `repo` as well if you want private contributions counted in the totals.
3. Expiry: your call. Whatever you pick, you'll be redoing this when it lapses, and the symptom will be the metrics workflow suddenly failing months from now.
4. Copy the token — this is the only time you'll see it.
5. **Repo → Settings → Secrets and variables → Actions → New repository secret.** Name it `METRICS_TOKEN`, exactly, and paste the value.

CLASSIC, NOT FINE-GRAINED

GitHub pushes you towards fine-grained tokens on that page, and a fine-grained token will not work here. If you take one thing from this section: it must be **Generate new token (classic)**. This single mistake accounts for most "the metrics workflow just won't run" problems.

The name matters too, because the workflows reference it literally:

YAML

```
token: ${ secrets.METRICS_TOKEN }
```

The cards workflow is more forgiving — it falls back gracefully:

YAML

```
GITHUB_TOKEN: ${ secrets.METRICS_TOKEN || secrets.GITHUB_TOKEN }
```

Meaning: use my token if it exists, otherwise use the built-in one and render fewer tiles. It degrades instead of failing, which is how this stuff should be written.

STEP 7

Push it, then light the fuse

TERMINAL

```
git add -A
git commit -m "profile readme"
git remote add origin https://github.com/YOUR_USERNAME/YOUR_USERNAME.git
git push -u origin main
```

Then go to the **Actions** tab. GitHub shows a "workflows aren't running on this fork / enable them" banner on a fresh repo — click through it. Then run each of the three manually via **Run workflow** rather than waiting six hours for the schedule.

First runs take a couple of minutes each. Metrics is the slowest because the isometric calendar is genuinely a lot of drawing.

The order that saves you a debugging session

- ☐ Repo is named exactly your username, and is **public**
- ☐ Workflow permissions set to **Read and write**
- ☐ `METRICS_TOKEN` secret added, from a **classic** token
- ☐ `metrics.yml` has your username in all four `user:` fields
- ☐ Both snake URLs in the README point at `YOU/YOU/output/`
- ☐ Every `YOUR_*` and `PROJECT_*` placeholder replaced
- ☐ Pushed to `main`
- ☐ All three workflows run once by hand, all three green
- ☐ Profile checked in dark mode *and* light mode
- ☐ Profile checked on your phone

After that first manual run, you're done. Metrics refreshes every six hours, the snake every twelve, the charts and cards once a day. The page keeps itself current with no further input from you — which is the actual point of the whole exercise.

A THING YOU'LL NOTICE

Your commit history will start filling with `chore: refresh charts and cards from github-actions[bot]`. That's normal and it's contained to this one repo. It doesn't count as contribution activity, so no, you can't farm your own graph this way.

STEP 8

When it breaks

It will break at least once. Here's the entire list, in rough order of how often it happened to me.

SYMPTOM	WHAT IT ACTUALLY IS
Images are broken for everyone but you	The repo is private. You're seeing them because you're authenticated. Make it public.
Images broken for you too	The files aren't pushed yet. Relative paths like <code>assets/portrait.svg</code> only resolve once the file exists on <code>main</code> .
Snake images 404	Either the snake workflow hasn't finished a run yet — normal on day one — or step 6A was skipped, so it couldn't create the <code>output</code> branch.
Metrics workflow fails	Almost always <code>METRICS_TOKEN</code> : missing, misspelled, expired, or created as fine-grained instead of classic.
Workflow succeeds but nothing changes	It couldn't push. Workflow permissions are still read-only — step 6A.
<code>command not found</code> inside a <code>run:</code> block	CRLF line endings. The runners are Linux; a Windows line ending at the end of a shell line becomes part of the command. This is exactly what <code>.gitattributes</code> in the repo prevents — don't delete it, and don't let an editor "helpfully" convert the YAML.
The typing banner is blank	Something in the <code>lines</code> parameter isn't URL-encoded. Spaces must be <code>+</code> , and an <code>&</code> must be <code>%26</code> or it silently cuts the URL in half.
A chart is invisible	You're looking at it in the theme it wasn't drawn for. The <code><picture></code> block is missing a <code>source</code> , or points at a file that didn't generate.
The language radar is one giant spike	Working as designed, badly. Lower <code>--curve</code> to <code>0.3</code> , and check whether a committed <code>dist/</code> or <code>node_modules/</code> is inflating one language — if so, add it to <code>--exclude</code> .
The portrait is a flat blob	<code>--equalize</code> is missing. Add it. If it's still muddy, cut the background out of the source photo.
Stat numbers look low	Private contributions aren't counted unless your token carries the <code>repo</code> scope. Regenerate the token with it ticked.
Everything worked, then stopped months later	Your token expired. That's the one maintenance task this setup has.

When a workflow fails, open the Actions tab, click the red run, and expand the step that failed. The error is nearly always in the last four lines, and it nearly always names the token or the permission.

Now make it not-mine

This is the step people skip, and it's the only one that determines whether your profile looks like yours or looks like a fork.

Pick one accent colour and be ruthless about it

Mine is `#39d353` — the brightest green in GitHub's contribution graph. It's in the typing banner, the radar fills, the card borders, the view counter, the snake. One colour, used everywhere, is what makes a page of unrelated generated images read as a single designed thing.

Change it and change it in all of those places. A page with three accent colours looks like three people made it.

Cut things

I generated a coding-habits card — commits by hour, commits by weekday — and then deleted it, because at the width it had to render it looked like a broken image rather than a chart. The version of this repo you cloned still generates it; it just isn't in the README.

Ship fewer widgets than you're capable of generating. Every one you add makes every other one matter less, and a profile you have to scroll for thirty seconds isn't read, it's skimmed and closed.

Have your own bit, or have none

The `~/` section headers, the fake `$ cat about.txt` prompt, the binary footer — that's a bit. It's a coherent one, it matches the terminal-green thing, and it's *mine*. Copied wholesale onto your page it will read as copied, because the person looking at your profile has probably already seen mine.

So take the mechanism and drop the costume. Some directions that aren't mine: numbered chapters like a book, a changelog, a museum placard, a spec sheet, a menu, a receipt. Or nothing at all — plain, well-spaced headings with a good portrait beat a mediocre theme every time.

Write the words like a person wrote them

The generated art gets attention for about four seconds. The whoami paragraph is what someone actually reads. Rewrite it until it sounds like how you'd describe your work out loud to somebody you liked. Then cut it by a third.

Go set your repo descriptions

Genuinely — go do this now. Every repo you feature should have a real one-line description on GitHub itself. It shows up in search, in your repo list, on the cards, and everywhere anyone finds you. It takes four minutes and almost nobody does it.

Check it small

Most people will open your profile on a phone. Two-column tables collapse, wide SVGs shrink until the labels are unreadable, and a 500 KB portrait takes real time on mobile data. Load your own profile on your phone before you tell

anyone about it.

THE ONLY RULE THAT MATTERS

A profile README is a claim about how much care you put into things nobody made you do. That's the whole signal. The dot matrix and the radars are just the evidence.

Everything on one page

Commands

TERMINAL

```
# setup
git clone https://github.com/gargibhardwaj24/gargibhardwaj24.git my-profile
cd my-profile && rm -rf .git && git init && git branch -M main
pip install pillow

# portrait - the one I shipped
python scripts/dotify.py me.png -o assets/portrait --cols 100 --equalize --detail 0.5 --color

# portrait - green monochrome, matching the contribution graph
python scripts/dotify.py me.png -o assets/portrait --cols 88 --equalize --detail 0.5 --animate

# portrait - literal 0s and 1s
python scripts/dotify.py me.png -o assets/portrait --mode binary --cols 62 --equalize --detail 0.5

# self-rated radar (edit assets/skills.json first)
python scripts/radar.py --data assets/skills.json -o assets/radar

# language radar, straight from the API
python scripts/radar.py --github YOU -o assets/radar-langs --limit 7 --values --curve 0.4 \
  --exclude "shell,makefile,dockerfile,batchfile,procfile"

# stat card + project cards (edit assets/projects.json first)
python scripts/cards.py --user YOU --out assets

# look at all of it before pushing
start preview.html          # macOS: open preview.html

# ship
git add -A && git commit -m "profile readme"
git remote add origin https://github.com/YOU/YOU.git
git push -u origin main
```

Files you edit

- `README.md` — the layout and all your words
- `assets/skills.json` — self-rated radar
- `assets/projects.json` — featured repos
- `.github/workflows/metrics.yml` — your username ×4, timezone

Files you don't

- `scripts/*.py` — the generators

Settings on GitHub

- Repo name = username, **public**
- Settings → Actions → General → **Read and write permissions**
- Settings → Secrets → Actions → `METRICS_TOKEN`
- Token from **github.com/settings/tokens, classic**, scope `read:user`

- `.gitattributes` — line endings, load-bearing
- `assets/*.svg` — all generated output

Where things come from

- shields.io — the badges
- simpleicons.org — badge logo names
- skillicons.dev — the tech strip
- readme-typing-svg.demolab.com — typing banner
- komarev.com/ghpvc — view counter
- lowlighter/metrics — calendar, languages, achievements
- Platane/snk — the snake

Built by Gargi Bhardwaj · the live version is at github.com/gargibhardwaj24 · if you build one from this, send it to me — I want to see what you did differently.